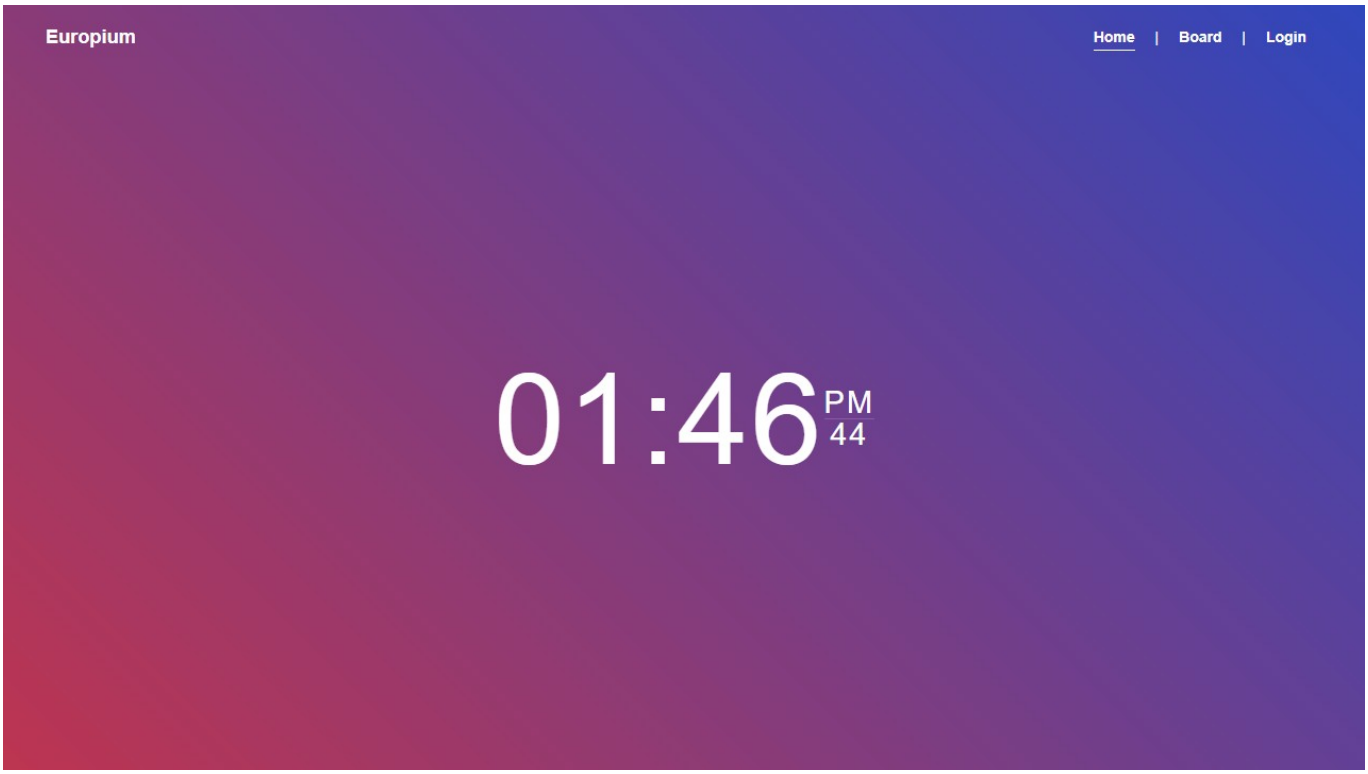
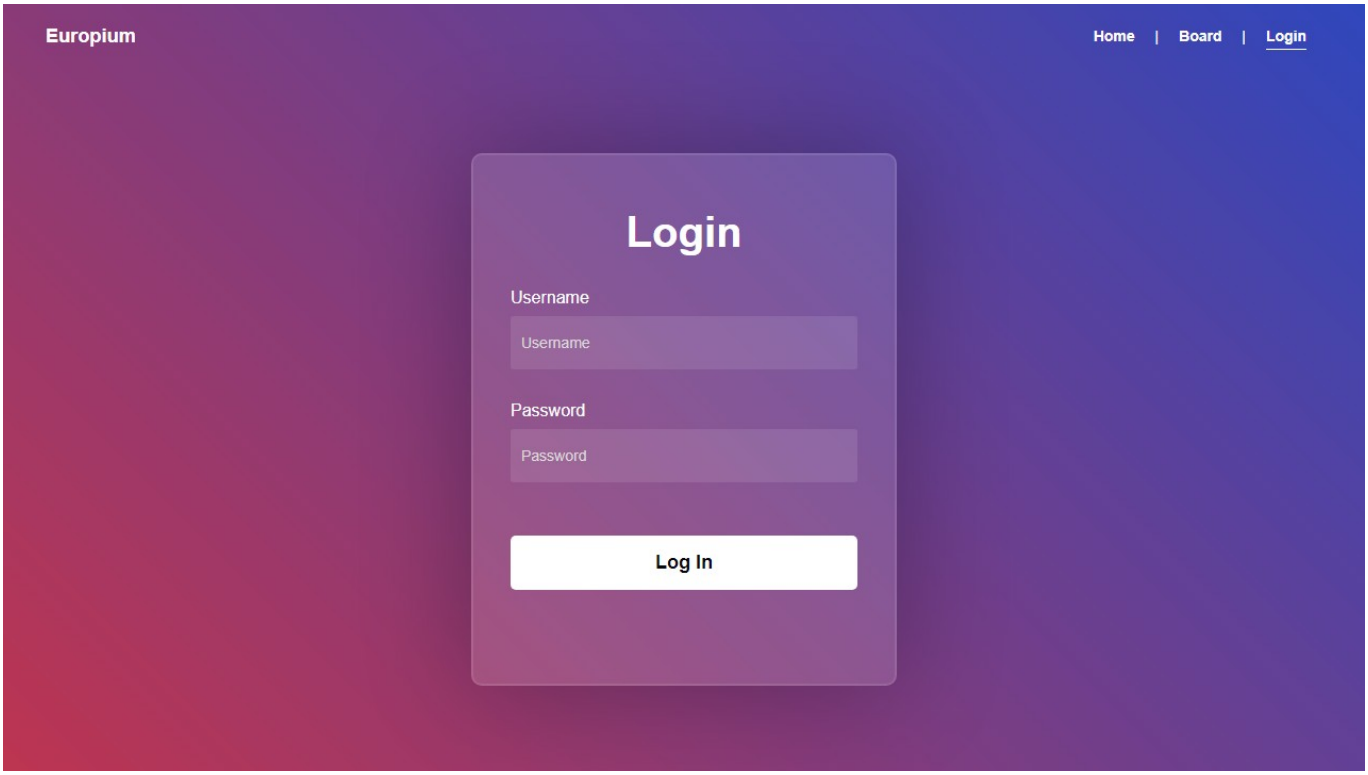
Front-end 과제





개요

- VUE(v2) + NODE 를 이용한 홈페이지 만들기

목표

- VUE(front-end server) 와 NODE(back-end sever) 를 이용하여 로그인과 게시판 기능이 있는 간단한 블로그 스타일 홈페이지를 개발한다.

과제 내용

- DB(데이터베이스)는 자유롭게 선택
  - mysql, postgree, mongodb, firebase..
- 로그인 시스템은 JWT 토큰 방식 이용
- "back-end sever"는 express 서버를 이용
  - 기존 express 경험자는 nestjs 를 사용하여 개발 (https://nestjs.com/)
- 게시판은 CRUD(create, read, update, delete) 기능 포함
- VUE 2 버전으로 개발
  - TYPESCRIPT 적용(class 스타일 적용 안함)
- VUE STORE 기능을 적극 이용
- 소스 형상관리는 GITHUB 이용
- 완성된 어플리케이션은 자신의 도메인 만들어서 인터넷에서 접근 가능하도록 발행.
- HTTPS 로 접근 가능하도록 한다

개발 환경

- node: `v16.13.2`
- npm: `v8.1.2`
- vscode: `v1.64.2`
- vue-cli: `v4.5.15`
- nest-cli: `v8.2.0`
- os: `windows 10`

빌드

- 인증서 재발급 방법

  인증서 발급 방법 openssl 설치 후

  ```
  openssl req -newkey rsa:4096 -days 365 -nodes -x509 -subj "/C=KR/ST=Seoul/L=Seoul/O=42Seoul/OU=Lee/CN=localhost" -keyout localhost.dev.key -out localhost.dev.crt
  ```

- 옵션 설명
  - req: 인증서명요청 또는 인증서를 만드는 커맨드
  - newkey rsa:4096: rsa 방식을 이용해 개인키와 인증서명 요청을 새로만든다. 4096 bit 사이즈의 키가 생성된다.
  - days 365: 유효일수
  - nodes: no des 를 줄인 것으로, 개인키를 비밀번호로 보호하고싶지 않을 때 사용. 이 옵션을 생략하면 매번 비밀번호 입력해야함.
  - x509: 이 옵션을 사용하면 인증서명요청 대신 Self Signed 인증서를 생성한다.

    - subj "/C=KR/ST=Seoul/L=Seoul/O=42Seoul/OU=Lee/CN=localhost": 인증서 안에 들어갈 정보를 명시해주는 부분
    - keyout <개인키 이름> : -keyout localhost.dev.key
    - out <인증서 이름> : -out localhost.dev.crt

생성된 인증서는 각각 server, front 폴더의 cert 폴더로 넣어줌

server

- 인증서
    - ormconfig.json
        - host: db 위치
        - port: db port
        - username: user 이름
        - password: user 패스워드
        - database: 데이터베이스 이름
        - entities: 엔티티 위치
            - dist를 바라보지않으면 읽지 못하는 오류가 있음

```
{
  "type": "mysql",
  "host": "dev-sys.openobject.net",
  "port": 3306,
  "username": "leeapril0612",
  "password": "open1404!",
  "database": "leeapril0612",
  "entities": ["./dist/**/*.entity.js"],
  "logging": true,
  "synchronize": true
}
```

- 실행
    - docker

```
$ docker build --tag las_server .
$ docker run -p 25000:3080 -d las_server
```

    - node

```
$ npm i
$ npm run start:dev
```

front

- docker

```
$ docker build --tag lsa_front .
$ docker run -p 25001:80 -p 25002:443 -d lsa_front
```

- docker-compose

```
$ docker-compose up -d
```

- node

```
$ npm i
$ npm run serve
```

Server

기존 `express` 사용 경험이 있음으로 `Nestjs`사용을 채택 하여 `NestJS cli`를 사용하여 프로젝트를 구성, `Typeorm`을 사용하여 orm을 적용하였으며 `repository 패턴`을 적용하였다.

- DataBase: MariaDB
- Language: JavaScript
- Framework: NestJS

NESTJS

`NestJS`는 `Express`와 같은 서버 프레임워크로, 테스트 기능, 확장석 느슨한 결합 및 유지관리가 용이한 어플리케이션을 만들수 있는 아키텍쳐를 제공한다.

디렉토리 구조는 다음과 같다.

```
├src
│  ├기능                  # 기능 별(Auth,user,board) 폴더
│  ├entities              # db entities
│  └util                  # 유틸
├ormconfig.json            # typeorm 설정
└package.json             # package.json
```

파일 설명

```
- *.controller          # 라우터 담당 controller
- *.module.ts           # module import 및 export 담당
- *.schema.ts           # joi를 사용한 validate 객체 모음
```

```
    - *.service.ts          # db 에 접근 후 데이터 가공
    - *.type.ts             # type 선언 모음 파일
    - *.repository.ts       # db 인터페이스
```

Util

- response.util.ts
  - 성공시 동일한 reponse를 주기위한 util 클래스
    - Default

```
{
  "code": 200,
  "data": "string"
}
```

Auth

jwt 토큰 생성 및 인증을 담당한다

JWT

Json 포맷을 이용하여 사용자에 대한 속성을 저장하는 Claim 기반의 Web Token이다. JWT는 토큰 자체를 정보로 사용하는 Self-Contained 방식으로 정보를 안전하게 전달한다. 주로 회원 인증이나 정보 전달에 사용된다 JWT는 header.payload.signature로 구성됨

- 생성된 토큰 예시


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwibmFtZSI6ImFkbWluI
iwibGFzdExvZ2luIjoiMjAyMi0wMi0yNVQwOToxOTo1Ni40NTNaIiwiaWF0IjoxNjQ1NzgwNzk2LCJleHA
iOjE2NDU3ODEwOTZ9.NON9Oe_fzmnpBvHK_4IJyAV2Y8SDnbiCKZRYjWXty-E
```


Passport

Passport.js는 Node.js를 위한 인증 미들웨어로. OAuth를 이용한 구글, 페이스북, 트위터, 혹은 전통적인 로컬 인증 방법(DB에 저장된 계정 정보로 로그인) 등 여러 가지 인증 방법을 제공하는데 이 개별 인증 방법을 Strategy 라고 부른다.

본 프로젝트에서는 @UseGuards 데코레이터를 사용한 컨트롤러의 사용자 검증 하는 역할을 한다

- jwt.strategy.ts

```
// Request에 Authorization cookie가 있는지 확인
const fromAuthCookie = function () {
  return function (request) {
    let token = null;
    if (request && request.cookies) {
```

```
      token = request.cookies["Authorization"];
    }
    return token;
  };
};

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private readonly authService: AuthService) {
    //
    super({
      jwtFromRequest: fromAuthCookie(),
      ignoreExpiration: false,
      secretOrKey: "lsa",
    });
  }

  async validate(payload: JwtPayload) {
    // payload에 맞는 user가 있는지 확인
    const user = this.authService.isUser(payload.username);
    if (!user) {
      return null;
    }
    // 존재시 Request에 user라는 이름의 payload정보를 넘겨줌
    return { ...payload };
  }
}
```

- local.strategy.ts
  - local.strategy는 username, password를 이용한 로그인 방식이다

```
@Injectable()
export class LocalStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: AuthService) {
    super();
  }

  async validate(username: string, password: string): Promise<LoginUserInfo>
{
    // username과 password가 일치하는 유저가 있는지 확인
    const user = await this.authService.validateUser(username, password);
    // 없으면 401
    if (!user) {
      throw new UnauthorizedException();
    }
    // 존재시 Request에 user라는 이름의 payload정보를 넘겨줌
    return user;
  }
}
```

- Request 로그에 확인되는 user

```
// Request 객체
{
  ...
  user: {
    username: 'admin',
    name: 'admin',
    lastLogin: 2022-03-02T00:53:58.103Z
  }
}
```

- 사용된 예
  - @nestjs/passport의 @UseGuards 데코레이터를 사용하여 passport 사용을 선언후 AuthGuard의 인자를 통해 검증에 사용할 strategy 지정 한다 @UseGuards(AuthGuard('local'))

```
//user.controller.ts
@Header('Content-Type', 'application/json')
@UseGuards(AuthGuard('local')) // local.strategy.ts
@Post('login')
public async login(@Res({ passthrough: true }) response: Response,
@Request() req): Promise<ResponseMessageBody> {
  ...
  return new ResponseMessage().success().body({
    token: access_token
  }).build();
}

@Header('Content-Type', 'application/json')
@UseGuards(AuthGuard('jwt')) // jwt.strategy.ts
@Post('register')
public async addUser(@Body() register: Register):
Promise<ResponseMessageBody> {
    ...
    return new ResponseMessage().success(201).body(user).build();
  } catch (err) {
    Logger.error(err);
  }
}
```

User

User의 기능은 login, logout, profile으로 구성된다

- LOGIN
  - description: 로그인 성공시 set-cookie header를 통해 Token을 http-only cookie로 set 함

        - http-only-cookie: CSS 취약점을 해결하는 방법으로 브라우저에서 Cookie접근을 차단하는 방식
    - Url: `/user/login`
    - Method: `Post`
    - Contnet-Type: `application/json`
    - Request Body

```
{
  "username": "string",
  "password": "string"
}
```

    - Resposne
        - Success
            - Header
                - set-Cookie: `Authorization`
            - Status: 200
            - Body

```
{
  "code": 200,
  "data": {
    "token": "string"
  }
}
```

        - Failed

```
{
  "statusCode": 401,
  "message": "Unauthorized"
}
```

- LOGOUT
    - Description: set 되어있는 쿠키를 삭제하는 api
    - Url: `/user/logout`
    - Method: `Get`
- Profile
    - Description: header에 토큰 유효 검사 후 토큰 유효시 해당 토큰에 유저 정보를 리턴
    - Url: `/user/profile`
    - Method: `Get`
    - Resposne

- Status: 200
- Body

```
{
  "data": [
    {
      "username": "string",
      "name": "string",
      "lastLoginDate": "2022-02-25T09:13:52.000Z",
      "createdDate": "2022-02-16T06:13:15.621Z"
    }
  ],
  "code": 200
}
```

Board

게시판 형식의 CRUD API

- List
  - Url: `/board/list`
  - Method: `get`
  - Resposne

```
{
  "data": [
    {
      "id": 13,
      "username": "admin",
      "title": "zxx",
      "content": "fdf",
      "createdDate": "2022-02-24T05:34:12.675Z",
      "updatedDate": "2022-02-24T05:34:12.675Z"
    }
    ...
  ],
  "code": 200
}
```

- getBoard
  - Url: `/board/{id}`
  - Method: `get`
  - Resposne

```
{
  "data": {
```

```
      "id": 13,
      "username": "admin",
      "title": "zxx",
      "content": "fdf",
      "createdDate": "2022-02-24T05:34:12.675Z",
      "updatedDate": "2022-02-24T05:34:12.675Z"
    },
    "code": 200
  }
```

- createBoard
  - Url: `/board/{id}`
  - Method: `post`
  - Contnet-Type: `application/json`
  - Request
    - header
      - cookie Authorization
    - body

```
{
  "title": "string",
  "content": "string"
}
```

  - Resposne
    - success

```
{
  "data": {
    "id": "number",
    "username": "string",
    "title": "string",
    "content": "string",
    "createdDate": "2022-02-24T05:34:12.675Z",
    "updatedDate": "2022-02-24T05:34:12.675Z"
  },
  "code": 201
}
```

    - failed

```
{
  "code": 403,
  "data": ""
}
```

- updateBoard
  - Url: `/board/{id}`
  - Method: `patch`
  - Contnet-Type: `application/json`
  - Request
    - header
      - cookie Authorization
    - body

```
{
  "title": "string",
  "content": "string"
}
```

  - Resposne
    - success

```
{
  "data": "",
  "code": 200
}
```

    - failed

```
{
  "code": 403,
  "data": ""
}
```

- deleteBoard
  - Url: `/board/{id}`
  - Method: `patch`
  - Contnet-Type: `application/json`
  - Request
    - header
      - cookie Authorization
    - body

```
{
  "title": "string",
```

```
    "content": "string"
  }
```

- Resposne
  - success

```
{
  "data": "",
  "code": 200
}
```

  - failed

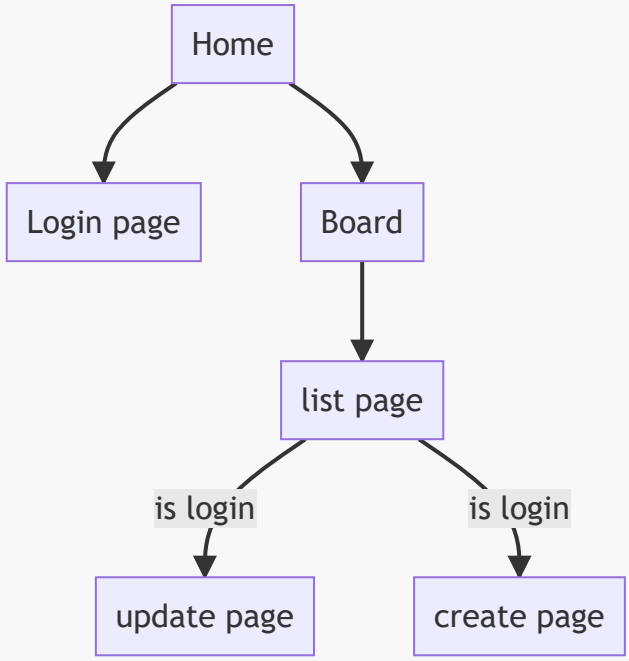
```
{
  "code": 403,
  "data": ""
}
```

Front

Vue cli를 사용하여 프로젝트를 구성

```
├public
├src
   ├assets
   ├components
   ├plugins
   ├router                  # 라우터
   ├service
   │  ├board                # board 서비스
   │  └user                 # user 서비스
   ├store
   │  ├auth                 # auth 스토어
   │  └board                # board 스토어
   ├util                    # 유틸
   │  └http                 # httprquest util
   └views
       ├board               # board
       │  └component        # board component
       └login               # login pages
```

화면 플로우



HttpRequest

공통 http 요청 처리를 위한 Axios wrapper

```
export class HttpRequest {
  private instance: AxiosInstance;

  constructor(option?: AxiosRequestConfig) {
    this.instance = axios.create({
      // api base url
      baseURL: "https://dev-sys.openobject.net:25000/",
      // set cookie를 위한 설정
      withCredentials: true,
      ...option,
    });

    this.instance.interceptors.response.use(undefined, (err) => {
      // 공통 에러 처리
      switch (err.response.status) {
        case 401:
          if (store.state.Auth.login) {
            store.dispatch("LOGOUT");
          }
          router.push("/login");
          break;
      }
      return Promise.reject(err);
    });
  }

  public request(option: AxiosRequestConfig): Promise<ReponseBody> {
    return this.instance.request(option).then((res) => res.data);
```

```
    }
  }
```

사용 예시: boardAPI.ts

```
export const getBoards = (option?: AxiosRequestConfig) => {
  return new HttpRequest().request({
    url: "/board/list",
    method: "get",
    headers: {
      "Content-Type": "application/json;charset=utf-8",
    },
    ...option,
  });
};
```

Login

로그인은 서버에서 HTTP 쿠키를 SET 해주는 방식을 사용

로컬스토리지에 login 유효 상태만을 저장하여 브라우져 종료시에도 로그인이 유지되도록 하였고 앱 최초 로드시 profile api를 호출 하여 토큰 유효 검사 및 profile 정보를 스토어에 저장하는 과정을 거쳐 토큰 유효 검사를 대체 하였음

AuthModule.ts

```
const Auth: Module<AuthState, RootState> = {
  state: {
    // 로그인 상태를 localStorage에 저장 true시 로그인중 최초 생성은 스토리지에서 가져
옴
    login: localStorage.getItem("login") === "true",
    // http cookie는 읽을 수 없어 payload를 읽을수 없음으로 profile을 저장하는 store
state 생성
    profile: null,
  },
  mutations,
  actions,
};
// mutations
const mutations: MutationTree<AuthState> = {
  LOGIN(state) {
    state.login = true;
    // 스토리지 로그인 상태 true로 변경
    localStorage.setItem("login", "true");
  },
  LOGOUT(state) {
    state.login = false;
    // 스토리지 아이템 리무브
    localStorage.removeItem("login");
```

```
  },
  setProfile(state, profile: UserInfo) {
    state.profile = profile;
  },
};

// action
const actions: ActionTree<AuthState, RootState> = {
  async LOGIN(State, { username, password }: LoginBody) {
    try {
      // 유효 검사 후 set cookie를 해주는 api
      await login({
        username,
        password,
      });
      // 성공시 profile을 가져오는 api 호출
      await this.dispatch("getProfile");
      return true;
    } catch (error) {
      return Promise.reject(error);
    }
  },

  async getProfile({ commit }) {
    try {
      const { data } = await getProfile();
      // 프로필 저장
      commit("setProfile", data);
      // 로그인 상태 변경 (mutations login)
      commit("LOGIN");
      return true;
    } catch (error) {
      return Promise.reject(error);
    }
  },
};
```

App.vue

```
export default {
  name: "app",
  created() {
    // 앱 최초 생성시 로그인 상태를 확인
    if (this.$store.state.Auth.login) {
      // profile 가져옴
      this.$store.dispatch("getProfile").catch(() => {
        // 실패시 토큰 미유효로 판단하여 로그아웃
        this.$store.dispatch("LOGOUT");
      });
    }
  },
};
```

Board

Board 인터페이스는 service/board/board.d.ts에 따로 선언하여 사용하였다

```
declare interface Board {
  id: number,
  username: string,
  title: string,
  content: string,
  createdDate: string,
  updatedDate?: string,
}

declare interface BoardBody {
  title?: string,
  content?: string
}
```

API인터페이스는 LOGIN과 마찬가지로 service/board/boardAPI.TS 파일에 정의 하여 사용 하였다

```
export const getBoards = (option?: AxiosRequestConfig) => {
  return new HttpRequest().request({
    url: '/board/list',
    method: 'get',
    headers: {
      'Content-Type': 'application/json;charset=utf-8'
    },
    ...option
  })
}

export const updateBoard = (id: number, body: BoardBody, option?:
AxiosRequestConfig) => {
  return new HttpRequest().request({
    url: `/board/${id}`,
    method: 'patch',
    headers: {
      'Content-Type': 'application/json;charset=utf-8'
    },
    data: body,
    ...option
  })
}

export const getBoard = (id: number) => {
  return new HttpRequest().request({
    url: `/board/${id}`,
    method: 'get'
```

```
  })
}

export const createBoard = (data: BoardBody, option?: AxiosRequestConfig) => {
  return new HttpRequest().request({
    url: '/board',
    method: 'POST',
    data: data,
    headers: {
      'Content-Type': 'application/json;charset=utf-8'
    },
    ...option
  })
}

export const deleteBoard = (id: number) => {
  return new HttpRequest().request({
    url: `/board/${id}`,
    method: 'delete'
  })
}
```

각 페이지에서 사용하는 데이터는 Board Store에 따로 state를 두어 사용하였다 /store/board/BoardModule.ts

```
export type BoardState = {
  boards: Board[],
  board: Board,


  // 로딩처리
  isLoadingBoards: boolean,
  isLoadingBoard: boolean,
  isLoadingCreateBoard: boolean,
  isLoadingDeleteBoard: boolean,
  isLoadingUpdateBoard: boolean,

  // 에러 처리
  errorBoards: string | null
  errorBoard: string | null,
  errorCreateBoard: string | null,
  errorDeleteBoard: string | null,
  errorUpdateBoard: string | null,


  // 완료 처리
  doneBoards: boolean,
  doneBoard: boolean,
  doneCreateBoard: boolean,
  doneDeleteBoard: boolean,
  doneUpdateBoard: boolean
}
```

```
const actions: ActionTree<BoardState, RootState> = {
  async GET_BOARDS ({ commit }) {
    commit('setIsLoadingBoards', true)
    commit('setDoneBoards', false)
    commit('setErrorBoards', null)
    const boards = this.state.Board.boards
    try {
      const { data } = await getBoards()
      commit('setBoards', [...boards, ...data])
      return Promise.resolve(true)
    } catch (error) {
      commit('setErrorBoards', error)
      return Promise.reject(error)
    } finally {
      commit('setIsLoadingBoards', false)
    }
  },

  ...
}
```

배포

docker를 이용한 배포

- 사용 이미지
  - server: node:16.4.2-alpine3.11
  - front: nginx:alpine
- 배포 URL
  - server: https://dev-sys.openobject.net:25000/
  - front: https://dev-sys.openobject.net:25002/

front

docker compose를 사용하여 배포, nginx.conf는 볼륨을 통하여 적용

https 인증서는 openssl을 이용하여 생성된 자체 서명 인증서를 사용함

- docker-compose.yml

```
version: "3.2"

services:
  lsa-front:
    container_name: lsa_front
    image: nginx:alpine
    ports:
      - "25001:80"
      - "25002:443"
    volumes:
      - ./dist:/usr/share/nginx/html
```

```
      - ./.nginx:/etc/nginx/conf.d
      - ./cert:/etc/ssl/
    networks:
      - elk

networks:
  elk:
    driver: bridge
```

- nginx.conf

```
  server {
    listen 80;
    listen [::]:80 ;
    client_max_body_size 5M;
    return 301 https://dev-sys.openobject.net:25002$request_uri;
  }

  server{
    listen 443 ssl;
    server_name _;
    ssl on;
    ssl_certificate /etc/ssl/localhost.dev.crt;
    ssl_certificate_key /etc/ssl/localhost.dev.key;
    ssl_prefer_server_ciphers on;
    location / {
      # proxy_pass http://tomcat/;

      # alias /usr/share/nginx/html/;
      root   /usr/share/nginx/html;
      try_files $uri $uri/ /index.html;

      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header Host $http_host;
    }
  }
```